

 Se former autrement	Année Universitaire : 2025-2026 Applications Web Distribuées UP_WEB
Workshop - Communication Asynchrone entre les Micro-services RabbitMQ	

Objectif

Fournir une compréhension approfondie de l'utilisation de **RabbitMQ** pour assurer une **communication asynchrone** entre les micro-services,

La **Communication asynchrone** : est une communication non bloquante où le service envoie un message sans attendre de réponse immédiate.

Utilisé pour **des traitements déconnectés** où les services peuvent fonctionner indépendamment et de manière parallèle. Généralement pour les tâches asynchrones, telles que les envois de courriels, les notifications, ou les tâches programmées

1. Principe de communication avec RabbitMQ :

Utilise un modèle de message connu par Message broker traditionnel basé sur des files d'attente permettant aux services de communiquer de manière asynchrone. Il repose sur le protocole **AMQP (Advanced Message Queuing Protocol)** et permet aux services d'échanger des messages via des **queues** (files d'attente) où les messages sont publiés par les producteurs et consommés par les consommateurs.

□ Avantages:

- Très adapté pour des communications asynchrones où les messages peuvent être traités indépendamment du moment où ils sont envoyés.
- Haute tolérance aux pannes : si le service est indisponible, RabbitMQ garde les messages en attente.
- Capacité à gérer de grandes quantités de messages.
- Découplage des micro-services, les services ne communiquent pas directement, facilitant la scalabilité et la maintenance.

❑ Inconvénients:

- Plus complexe à configurer et à gérer que des appels REST simples.
- Nécessite la gestion des états des messages.

2. RabbitMQ : Concepts, Déclaration et Gestion des Queue

a. Les Concepts Clés de RabbitMQ :

Broker (RabbitMQ)

- RabbitMQ est un **Broker** de message.
- Il agit comme un intermédiaire qui reçoit, stocke et transmet les messages (dans une queue) entre les producteurs et les consommateurs.
- Il garantit que les messages ne sont pas perdus et qu'ils seront livrés au bon consommateur.
- Les producteurs et les consommateurs n'ont pas besoin de se connaître directement.

Producer (Producteur)

- C'est le service qui **envoie des messages** à RabbitMQ.

Consumer (Consommateur)

- C'est le service qui **récupère et traite** les messages provenant de RabbitMQ.

Queue (File d'attente)

- C'est l'endroit où RabbitMQ stocke les messages avant qu'ils soient consommés.
- Les messages sont livrés aux consommateurs dans l'ordre FIFO (**First In, First Out**).
- Une queue ne peut être consommée que par les consommateurs qui y sont abonnés.
- Si aucun consommateur n'est disponible, le message reste dans la queue.

b. Qui doit déclarer la queue RabbitMQ: Consumer ou Producer ?

Scénario 1 – Queue déclarée côté producteur

- Si le producteur plante au démarrage, la queue ne sera pas créée → le consommateur ne peut rien écouter.
- Le producteur a la responsabilité de configurer quelque chose qui n'est pas pour lui.

Scénario 2 – Queue déclarée côté consommateur (meilleure pratique)

- Le consommateur crée sa queue s'il ne la trouve pas.

- Le producteur n'a pas besoin de connaître les paramètres de la queue, seulement son nom.
- Cela respecte le principe de responsabilité unique → chaque microservice gère ce dont il a besoin.
- Si demain un autre consommateur apparaît (un autre microservice intéressé par les mêmes événements du producteur), **il peut déclarer sa propre queue**, indépendante du premier consommateur.

→ **Conclusion : Le consommateur déclare la queue, le producteur se contente de publier en utilisant simplement le nom**

- Le consommateur est le meilleur endroit pour déclarer la queue.
- Le producteur peut publier même si personne ne consomme.

c. Que fait RabbitMQ si la queue n'existe pas ?

- Si le producteur envoie un message sur une queue inexistante → **RabbitMQ rejette le message** avec une erreur (404 NOT_FOUND).
- D'où l'intérêt que **au moins un service** déclare la queue avant que le producteur ne publie.

3. Mise en œuvre de RabbitMQ pour les Microservices : Étapes et Tests

Étape 1 : Installation & Configuration de RabbitMQ

1. Installer Erlang :

Erlang est un environnement d'exécution conçu pour créer des systèmes distribués, tolérants aux pannes, et hautement concurrentiels. De plus, le code source de RabbitMQ est écrit en Erlang, et il utilise les fonctionnalités de l'environnement d'exécution Erlang pour sa propre exécution.

- Téléchargez l'installateur Erlang pour Windows depuis votre espace blackboard ou bien [le site officiel d'Erlang](#).
- Exécutez l'installateur et suivez les instructions à l'écran.

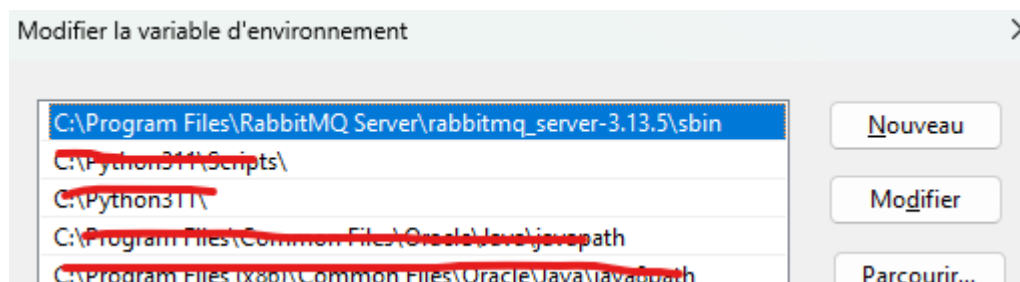
NB : RabbitMQ nécessite impérativement Erlang pour fonctionner. **Si Erlang n'est pas installé, RabbitMQ ne pourra pas démarrer.** Voici les raisons :

- **Absence de l'Environnement d'Exécution** : RabbitMQ dépend de l'environnement d'exécution d'Erlang pour exécuter ses processus et gérer les messages. **Sans Erlang, cet environnement n'existe pas.**
- **Échecs de Démarrage** : Lors du démarrage de RabbitMQ, il cherchera à utiliser les bibliothèques et les services fournis par Erlang. **Si Erlang n'est pas installé, RabbitMQ échouera à démarrer et générera des erreurs indiquant que l'environnement Erlang est manquant.**

2. Installer RabbitMQ :

Une fois Erlang installé, vous pouvez installer RabbitMQ en respectant les instructions suivantes :

- o Téléchargez l'installateur RabbitMQ pour Windows depuis votre espace blackboard ou bien [le site officiel de RabbitMQ](#).
- o Exécutez l'installateur et suivez les instructions à l'écran.
- o Ajoutez le chemin **C:\Program Files\RabbitMQ Server\rabbitmq_server-x.x.x\sbin** (ou l'emplacement où RabbitMQ a été installé) à la variable d'environnement PATH.



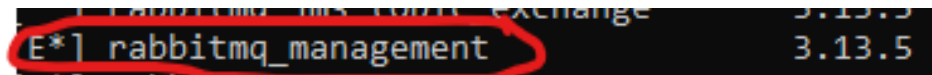
3. Démarrer RabbitMQ :

- o Ouvrez une invite de commande en tant qu'administrateur.
- o Exécutez **rabbitmq-plugins enable rabbitmq_management** pour activer le plugin de gestion web.
- o Exécutez **rabbitmq-service start** pour démarrer le service RabbitMQ.
- o Pour avoir des informations sur l'état de RabbitMQ et les processus actifs, exécutez **rabbitmqctl status**.

4. Accéder à l'Interface de Gestion Web :

- o Assurez-vous que le service RabbitMQ est en cours d'exécution et accessible via l'interface de gestion à l'adresse <http://localhost:15672>. Le port **15672** est utilisé par défaut pour l'interface web d'administration de RabbitMQ
- o Les identifiants par défaut sont : **Username** : guest et **Password** : guest
- o Si vous avez rencontré un problème d'accès à la page web de Rabbit :
 - **Vérifier si le plugin rabbitmq_management est Activé** via cette commande : **rabbitmq-plugins list**

Recherchez `rabbitmq_management` dans la liste. Il devrait être précédé d'un `E` (Enabled) :



- Activer le plugin si nécessaire : `rabbitmq-plugins enable rabbitmq_management`
- Redémarrez RabbitMQ après avoir activé le plugin :
- `rabbitmq-service.bat stop`
 - `rabbitmq-service.bat start`

Étape 2 : Configuration & Implémentation des Microservices

Dans notre exemple, on va assurer la communication entre le service Candidat qui appelle le service Job pour enregistrer les jobs dans la liste de favoris.

Le flux de communication avec RabbitMQ est le suivant :

❖ Déclaration de la queue (Côté Consommateur – Candidat)

- Le consommateur crée et déclare la queue `CANDID_JOB_QUEUE` dans RabbitMQ.
- Cela garantit que la queue existe avant que le producteur n'envoie un message.
- Le consommateur s'abonne à cette queue via `@RabbitListener`, ce qui le rend prêt à recevoir de futurs messages.

❖ Publication du message par le Producteur (Job)

- Le service Job convertit l'objet `JobDTO` en JSON et publie ce message vers la queue déjà créée `CANDID_JOB_QUEUE`.
- Le producteur ne déclare pas la queue — il suppose qu'elle existe (grâce au consommateur).
- RabbitMQ reçoit le message et l'ajoute dans la file.

❖ Stockage du message dans RabbitMQ

- RabbitMQ conserve le message dans la queue `CANDID_JOB_QUEUE`.
- Si aucun consommateur n'est en ligne, le message reste stocké dans la queue jusqu'à ce qu'un consommateur soit disponible.
- L'ordre de traitement est respecté (FIFO).

❖ Consommation et traitement du message (Côté Candidat)

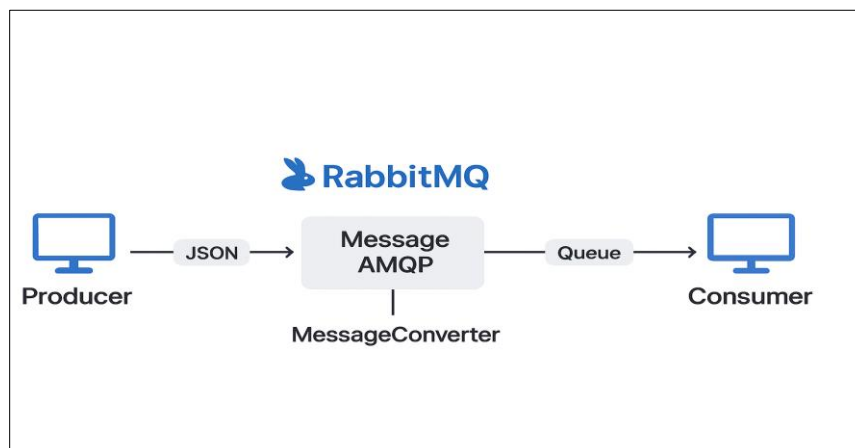
- Comme le consommateur est abonné, il est notifié automatiquement qu'un message est disponible.
- Il le lit, le convertit en objet `JobDTO` et exécute sa logique métier

❖ Orchestration Envoi/ Réception de Messages (RabbitTemplate)

- RabbitTemplate est la classe principale fournie par Spring AMQP pour interagir avec RabbitMQ.
- Il représente un point d'entrée pour :
 - Envoyer des messages (convertAndSend, send)
 - Recevoir des messages (receive, receiveAndConvert)
- Il gère :
 - La connexion au broker RabbitMQ
 - La création de canaux AMQP
 - L'utilisation du **MessageConverter** pour la sérialisation/désérialisation

❖ Conversion Objet ↔ Message AMQP (MessageConverter)

- Le **MessageConverter** en Spring AMQP est une stratégie de conversion utilisée par RabbitTemplate.
- Appelé **automatiquement (pas besoin de l'injecter)** par Spring AMQP:
 - **Avant l'envoi** d'un objet (sérialisation)
 - **Après la réception** d'un message (désérialisation)
- Ne travaille pas directement avec un "objet JSON", mais bien avec un **Message AMQP**.
- Son travail officiel est de convertir :
 - **À l'envoi (Producer)** : Object Java (ex: JobDTO) → Message AMQP
 - **À la réception (Consumer)** : Message AMQP → Object Java (ex: JobDTO)
- C'est **Jackson2JsonMessageConverter** qui assure la conversion suivante : Objet Java → JSON → AMQP Message et inversement.



Nous allons suivre ces étapes d'implémentation :

A. Configuration de RabbitMQ dans les deux micro-services.

1. Ajouter la dépendance Spring **AMQP** (Advanced Message Queuing Protocol) pour RabbitMQ dans chaque `pom.xml` :

```
<!-- Spring for RabbitMQ -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-amqp</artifactId>
</dependency>
```

NB : Lorsque vous configurez **Spring Boot** pour utiliser RabbitMQ, votre application communique avec RabbitMQ via le **protocole AMQP**.

2. Ajouter ces configurations RabbitMQ dans le fichier `application.properties` approprié à chaque micro-service:

```
# Spring RabbitMQ
spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672 # port par défaut du protocole AMQP
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest
```

NB : Port **5672** est le **port par défaut** du protocole AMQP (celui que RabbitMQ utilise pour la communication entre applications et broker). Le Producer et le Consumer doivent se connecter **au même broker**, donc à la même adresse et au même port.

3. Configuration RabbitMQ (**Candidat — consommateur**) : Créer la classe `RabbitMQConfig.java`.

But : déclarer la queue `jobQueue` (`durable = true`) et configurer le `MessageConverter` pour désérialiser automatiquement en `JobDTO`.

```
package com.esprit.ms.candidat;

import org.springframework.amqp.core.Queue;
import org.springframework.amqp.rabbit.config.SimpleRabbitListenerContainerFactory;
import org.springframework.amqp.rabbit.connection.ConnectionFactory;
import org.springframework.amqp.support.converter.Jackson2JsonMessageConverter;
import org.springframework.amqp.support.converter.MessageConverter;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class RabbitMQConfig {

    // Nom de la queue
    public static final String CANDID_JOB_QUEUE = "jobQueue";
```

```

// Déclarer la queue durable (true) pour survivre au redémarrage du broker
@Bean
public Queue jobQueue() {
    return new Queue(CANDID_JOB_QUEUE, true);
}

// Converter JSON <-> POJO pour que Spring transforme automatiquement les
messages
@Bean
public MessageConverter jsonMessageConverter() {
    return new Jackson2JsonMessageConverter();
}

// Factory utilisée par @RabbitListener pour utiliser le converter
@Bean
public SimpleRabbitListenerContainerFactory rabbitListenerContainerFactory(
    ConnectionFactory cf, MessageConverter converter) {
    SimpleRabbitListenerContainerFactory factory = new
SimpleRabbitListenerContainerFactory();
    factory.setConnectionFactory(cf);
    factory.setMessageConverter(converter);
    factory.setConcurrentConsumers(1);
    factory.setMaxConcurrentConsumers(3);
    return factory;
}
}

```

4. Configuration RabbitMQ (JobMS — producteur): Créer la classe RabbitMQConfig.java

But : configurer RabbitTemplate avec Jackson2JsonMessageConverter (pour envoyer directement l'objet), **sans** déclarer la queue (levé par le consommateur).

```

package com.esprit.jobms;

import org.springframework.amqp.rabbit.connection.ConnectionFactory;
import org.springframework.amqp.rabbit.core.RabbitTemplate;
import org.springframework.amqp.support.converter.Jackson2JsonMessageConverter;
import org.springframework.amqp.support.converter.MessageConverter;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class RabbitMQConfig {

    // Nom de la queue SANS avoir besoin à déclaré la queue
    public static final String CANDID_JOB_QUEUE = "jobQueue";

    // sérialisation JSON <-> POJO automatique pour RabbitTemplate
    @Bean
    public MessageConverter jsonMessageConverter() {
        return new Jackson2JsonMessageConverter();
    }

    // RabbitTemplate configuré pour utiliser le converter
    @Bean

```



```

    public RabbitTemplate rabbitTemplate(ConnectionFactory connectionFactory,
    MessageConverter converter) {
        RabbitTemplate template = new RabbitTemplate(connectionFactory);
        template.setMessageConverter(converter);
        return template;
    }
}

```

B. Mise en place d'un modèle de message DTO pour transférer les données

Créer une classe modèle (DTO) `JobDTO.java` pour Job dans les deux micro-services (Job et Candidat):

```

// DTO simple utilisé pour l'échange RabbitMQ

public class JobDTO {
    private int id;
    private String service;
    private boolean etat;

    public int getId() { return id; }

    public void setId(int id) { this.id = id; }

    public String getService() { return service; }

    public void setService(String service) {
        this.service = service;
    }

    public boolean isEtat() {
        return etat;
    }

    public void setEtat(boolean etat) {
        this.etat = etat;
    }
}

```

- Cette classe **DTO (Data Transfer Object)** va représenter les informations que nous transférons entre les micro-services via RabbitMQ.
- Au lieu de créer une entité qui représente le job, on utilise un **DTO** dans le service Candidat pour recevoir les données du service Job. Cela vous permet de désérialiser le message reçu sans lier directement les données à une entité de base de données.
- Même si une entité Job existe déjà dans le micro-service Job, la classe JobDTO sera ajoutée.

Avantages :

- Séparer données persistées et données échangées

@Entity Job	JobDTO
- Représente la structure de la base de données. - Contient tous les champs nécessaires pour le stockage (ex : id, createdAt, relations avec d'autres entités, etc.)	- Représente les données à partager avec un autre microservice via RabbitMQ ou une API.

- Garantir une Indépendance entre les micro-services

L'entité Job peut évoluer indépendamment dans le micro-service Job sans affecter directement le contrat d'API utilisé pour échanger des données avec d'autres services. Cela permet une certaine flexibilité dans l'évolution des services.

C. Création du producteur (JobProducer) : envoi des messages à RabbitMQ

Créer la classe `JobProducer.java` dans le projet du service Job.

But : envoyer l'objet `JobDTO` directement vers la queue `CANDID_JOB_QUEUE` en utilisant la méthode `RabbitTemplate.convertAndSend()`.

Cette dernière fait deux choses en une seule étape :

1. **Convertir** ton objet en message AMQP en utilisant le `MessageConverter` (ici → JSON).
2. **Envoyer** ce message à la queue cible dans RabbitMQ.

```
package com.esprit.jobms;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.amqp.AmqpException;
import org.springframework.amqp.rabbit.core.RabbitTemplate;
import org.springframework.stereotype.Service;

@Service
public class JobProducer {

    private final RabbitTemplate rabbitTemplate;
    private static final Logger log = LoggerFactory.getLogger(JobProducer.class);

    // Inject RabbitTemplate (configuré avec Jackson converter)
    public JobProducer(RabbitTemplate rabbitTemplate) {
        this.rabbitTemplate = rabbitTemplate;
    }
}
```

```

public void sendJob(JobDTO job) {
    try {
        rabbitTemplate.convertAndSend(RabbitMQConfig.CANDID_JOB_QUEUE, job);
        log.info("Job envoyé à la queue : ", RabbitMQConfig.CANDID_JOB_QUEUE, job);
    } catch (AmqpException e) {
        log.error("Erreur lors de l'envoi du job à RabbitMQ", e);
        // Remonter l'exception pour que l'appelant sache que l'envoi a échoué
        throw e;
    }
}
}

```

NB :

- Utiliser le même nom de queue déclaré par le consommateur.
- Si le producteur démarre avant le consommateur (et que la queue n'existe pas), l'envoi échouera.

D. Service métier côté producteur (JobService) - appel du producteur après sauvegarde:

```

package com.esprit.jobms;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.util.List;
import java.util.Optional;

@Service
public class JobService {
    private final JobRepository jobRepository;
    private final JobProducer jobProducer;
    private static final Logger log = LoggerFactory.getLogger(JobService.class);

    public JobService(JobRepository jobRepository, JobProducer jobProducer) {
        this.jobRepository = jobRepository;
        this.jobProducer = jobProducer;
    }

    /** Sauvegarde d'un Job dans la base de données et envoi d'un DTO à RabbitMQ */

    //si save réussit mais l'envoi échoue, on aura un enregistrement sans message
    @Transactional
    public Job saveAndSendJob(Job jobEntity) {
        Job savedJob = jobRepository.save(jobEntity);
        log.info("Job sauvegardé en base : {}", savedJob.getService());

        // Construire un DTO à envoyer
        JobDTO jobDTO = new JobDTO(savedJob.getId(), savedJob.getService(), savedJob.isEtat());

        // Envoi asynchrone via RabbitMQ
        jobProducer.sendJob(jobDTO);

        return savedJob;
    }
}

```

```

    }

    public Optional<Job> getJobById(int id) {
        return jobRepository.findById(id);
    }

    public List<Job> getAllJobs() {
        return jobRepository.findAll();
    }
}

```

E. Création du consommateur (JobConsumer) : reçoit et traite les messages.

Créer la classe `JobConsumer.java` dans le projet du service Candidat.

But : consommer l'objet `JobDTO` automatiquement typé et déléguer le traitement à `CandidatService`.

```

package com.esprit.ms.candidat;

import com.fasterxml.jackson.databind.ObjectMapper;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.amqp.rabbit.annotation.RabbitListener;
import org.springframework.stereotype.Service;

@Service
public class JobConsumer {

    private final CandidatService candidatService;
    private static final Logger log = LoggerFactory.getLogger(JobConsumer.class);

    public JobConsumer(CandidatService candidatService, ObjectMapper objectMapper) {
        this.candidatService = candidatService;
    }

    // Ecoute la file CANDID_JOB_QUEUE
    // Spring convertit automatiquement le JSON en JobDTO grâce au MessageConverter

    @RabbitListener(queues = RabbitMQConfig.CANDID_JOB_QUEUE, containerFactory =
"rabbitListenerContainerFactory")
    public void receiveJob(JobDTO jobDTO) {
        log.info("JobDTO reçu depuis RabbitMQ : {}", jobDTO);
        // Déléguer la logique métier
        candidatService.receiveJobService(jobDTO);
    }
}

```

Ici, Spring Rabbit :

1. Reçoit le message de la queue
2. Utilise implicitement le **MessageConverter** pour désérialiser le JSON en JobDTO
3. Passe l'objet déjà converti à ta méthode `receiveJob()`

F. Service métier côté consommateur (CandidatService) – Traitement

```
package com.esprit.ms.candidat;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.util.ArrayList;
import java.util.List;

@Service
public class CandidatService {

    @Autowired
    private CandidatRepository candidatRepository;

    // Simulons un cache en mémoire pour les jobs reçus
    private List<JobDTO> favoriteJobDTOS = new ArrayList<>();
    private static final Logger log = LoggerFactory.getLogger(CandidatService.class);

    public void receiveJobService(JobDTO jobDTO) {
        log.info("Traitement du jobDTO received : {}", jobDTO.getService());
        addJobToFavorites(jobDTO);
        sendNotificationToUser(jobDTO);
    }

    private void addJobToFavorites(JobDTO jobDTO) {
        // Ajoutons le JobDTO à la liste en mémoire (cachée)
        favoriteJobDTOS.add(jobDTO);
        log.info("JobDTO ajouté aux favoris : {}", jobDTO.getService());
    }

    private void sendNotificationToUser(JobDTO jobDTO) {
        // Simulation de l'envoi d'une notification
        log.info("Notification envoyée au candidat: Nouveau jobDTO - {}",
jobDTO.getService());
    }

    // Simulez l'affichage des jobs favoris depuis le cache
    public List<JobDTO> getFavoriteJobs() {
        return favoriteJobDTOS;
    }

    public List<Candidat> getAllCandidats() {
        return candidatRepository.findAll();
    }
}
```

G. Endpoint de test côté producer (JobRestAPI)

Ajouter la méthode `createJob()` ci-dessous :

```
package com.esprit.jobms;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.Optional;

@RestController
@RequestMapping("/jobs")
public class JobRestAPI {

    private final JobService jobService;
    private String hello="Hello, i'm the Job MS";

    @Autowired
    public JobRestAPI(JobService jobService) {
        this.jobService = jobService;
    }

    @RequestMapping("/helloJ")
    public String sayHello(){
        return hello;
    }

    @GetMapping
    public List<Job> getAllJobs() {
        return jobService.getAllJobs();
    }

    @PostMapping("/send")
    public ResponseEntity<Job> createJob(@RequestBody Job job) {
        Job savedJob = jobService.saveAndSendJob(job);
        return ResponseEntity.ok(savedJob);
        // return "Job created and sent to RabbitMQ with ID: " +
        savedJob.getId();
    }

    @RequestMapping("/{id}")
    public Optional<Job> getJobById(@PathVariable int id) {
        return jobService.getJobById(id);
    }
}
```

H. Endpoint de test côté consommateur (CandidatRestAPI)

Ajouter la méthode `getFavoriteJobs()` ci-dessous :

```
package com.esprit.ms.candidat;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import java.util.List;

@RequestMapping("/candidats")
@RestController
public class CandidatRestAPI {

    private final CandidatService candidatService;

    @Autowired
    public CandidatRestAPI(CandidatService candidatService) {
        this.candidatService = candidatService;
    }

    private String title="Hello, i'm the 5SE MS";

    @RequestMapping("/hello")
    public String sayHello(){
        System.out.println(title);
        return title;
    }

    @GetMapping
    public List<Candidat> getAllCandidats() {
        return candidatService.getAllCandidats();
    }

    @GetMapping("/favorites")
    public List<JobDTO> getFavoriteJobs() {
        return candidatService.getFavoriteJobs();
    }
}
```

I. Tester l'application

1. Démarrer RabbitMQ :
 - Assurez-vous que RabbitMQ est en cours d'exécution.
2. Lancer Eureka
3. Lancer le Microservices Candidat **en premier** pour qu'il crée la queue `CANDID_JOB_QUEUE`

- Si tu démarres JobMS avant Candidat, et que jobQueue n'existe pas, rabbitTemplate.convertAndSend lèvera une exception (404)

4. Démarrer le service Job

5. Créer un job via l'API REST de Job :



The screenshot shows a REST client interface. At the top, the method is set to 'POST' and the URL is 'http://localhost:8073/jobs/send'. Below this, there are tabs for 'Params', 'Auth', 'Headers (9)', 'Body', 'Scripts', and 'Settings'. The 'Body' tab is selected, and the format is set to 'JSON'. The body content is a JSON object: `{"service": "Software Engineer", "etat": true}`. On the right side, there are buttons for 'Send', 'Cookies', and 'Beautify'.

6. Vérifier que Candidat reçoit bien le message dans la console et ajoute le job à la liste des favoris :

- Sur la console du Service Candidat, vous devriez voir le log :

```
INFO ... JobDTO reçu depuis RabbitMQ
INFO ... Traitement du jobDTO received : Software Engineer
INFO ... JobDTO ajouté aux favoris : Software Engineer
INFO ... Notification envoyée au candidat: Nouveau jobDTO - Software Engineer
```

7. Vérifie dans l'UI RabbitMQ (<http://localhost:15672>) : onglets *Queues* si la queue est présente :

localhost:15672/#/queues

Refreshed 2025-09-15 02:00:14 Refresh every 5 seconds

RabbitMQ 27.0.1 RabbitMQ 3.13.5 Erlang

Virtual host All Cluster rabbit@DESKTOP-62GT3 User guest Log out

All stable feature flags must be enabled after completing an upgrade. [Learn more]

Overview Connections Channels Exchanges **Queues and Streams** Admin

Queues

All queues (1)

Pagination

Page 1 of 1 - Filter: ☐ Regex ? Displaying 1 item , page size up to: 10

Overview					Messages			Message rates		
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver / get	ack
/	jobQueue	classic	D	idle	0	0	0	0.00/s	0.00/s	0.00/s

8. Afficher la liste de Job favoris enregistrés dans le service Candidat :

GET http://localhost:8072/candidats/favorites Send

Params Auth Headers (7) Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit

Body 200 OK - 118 ms - 216 B

{ } JSON Preview Visualize

```

1  [
2    {
3      "id": 3,
4      "service": "Software Engineer",
5      "etat": true
6    }
7  ]

```